

浙江大学

Zhejiang University-University of Illinois Urbana-Champaign Institute

Undergraduate Thesis (Design)



Thesis Title	Adaptive Federated Learning Over the Air
Student Name	Peiran Wei
Student ID	3220111228
Supervisor	Hao Yang
Class of year	2026
Major	ECE
Submission date	2026.05.22

Content Checklist

(Student should bind this form on the first page of the thesis after the cover page. The form should be filled out by the supervisor after the oral defense.)

Student Name	Peiran Wei	ZJU ID	3220111228	Major	ECE	
Supervisor	Hao Yang			Academic Title	Assistant Professor	
Thesis title	Adaptive Federated Learning Over the Air					
Content Checklist					Evaluation Section (√)	
Senior Thesis	1. Cover (Use the unified cover, blue recommended)				Good	Fair
	2. Academic Integrity Commitment of Senior Thesis					
	3. Acknowledgements					
	4. Abstract					
	5. Table of Contents					
	6. Main Content (Page numbers start from the main text)					
	7. References					
	8. Appendix (if needed)					
	9. Author's Biography					
	10. Task plan of senior thesis					
	11. Assessment form of project proposal					
	12. Assessment Form of senior thesis					
	13. Review Form of senior thesis					
	14. On-site Minutes Form of Oral Defense					
Signature of Supervisor:				Date:		

**ZJU-UIUC INSTITUTE**Zhejiang University-University of Illinois Urbana-Champaign Institute
浙江大学伊利诺伊大学厄巴纳香槟校区联合学院

Academic Integrity Commitment of Senior Thesis

1. I solemnly promise that the submitted thesis has been completed under the guidance of my supervisor in strict accordance with the relevant regulations of the university and the college.
2. In my thesis, except for the parts specifically marked and acknowledged, it does not contain research results that have been published or written by others, nor does it contain materials used to obtain degrees or certificates from Zhejiang University or other educational institutions.
3. Any contributions made by my colleagues to this research have been clearly stated and acknowledged in the thesis.
4. I promise that there has been no fabrication of data in the process of completing the thesis.
5. If there is any infringement of intellectual property rights in this thesis, I shall bear the corresponding legal responsibilities.
6. I fully understand that Zhejiang University has the right to retain and submit copies and disks of this thesis (design) to relevant departments or agencies, and to allow the thesis (design) to be consulted and borrowed. I authorize Zhejiang University to include all or part of this thesis (design) in relevant databases for retrieval and dissemination, and to save and compile this thesis (design) by means of photocopying, reduction, or scanning.

Author's Signature:

Supervisor's Signature:

Date:

Date:

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Hao Yang, for his guidance, feedback, and support throughout this project. His advice helped shape the technical direction of the work and kept the thesis focused on a clear and achievable research goal.

I am also grateful to the ZJU-UIUC Institute for providing the academic environment and course support that made this project possible. Finally, I thank my family, friends, and classmates for their encouragement during the development, experimentation, and writing stages of this thesis.

Abstract

Federated learning enables distributed model training without moving raw client data, but conventional digital aggregation requires communication resources that grow with the number of participating clients. Analog over-the-air federated learning (OTA-FL) addresses this bottleneck by letting client updates superpose directly in the wireless multiple-access channel. The same physical superposition that makes OTA-FL scalable also injects channel noise and impulsive interference into the aggregated update. This is especially challenging when the interference is heavy-tailed: rare large perturbations can dominate a server update and destabilize plain FedAvg-style training.

This thesis studies a PyTorch simulation framework for adaptive federated learning over the air under symmetric alpha-stable interference. The AWGN case is treated as the special case $\alpha = 2$, while $\alpha < 2$ models heavier-tailed channel perturbations. The central hypothesis is that server-side adaptive optimizers, especially AdaGrad-OTA and Adam-OTA, act as implicit channel-noise filters: when the noisy aggregate has large coordinate-wise magnitude, the fractional second-moment accumulator increases and the effective learning rate automatically contracts.

The framework implements FedAvg-OTA, FedAvgM-OTA, AdaGrad-OTA, and Adam-OTA; modular dataset partitioning; a noisy OTA aggregation oracle based on the Chambers-Mallows-Stuck alpha-stable sampler; Median Anchored Clipping (MAC) as robust pre-processing; JSON result export; and plotting scripts. The revised experiments prioritize CIFAR-10 tail-index ablations and MAC comparisons, with MNIST used as a lightweight sanity benchmark. The project is not a full line-by-line reproduction of the reference paper, but it implements the core heavy-tail setting needed to evaluate whether adaptive server normalization improves OTA-FL robustness.

Keywords: federated learning; over-the-air aggregation; alpha-stable noise; heavy-tailed interference; adaptive optimization; MAC

Content

Chapter 1. Introduction	2
1.1 Motivation	2
1.2 Problem Statement	3
1.3 Contributions	3
1.4 Thesis Organization	4
Chapter 2. Background and Related Work	5
2.1 Federated Learning	5
2.2 Analog Over-the-Air Aggregation	5
2.3 Adaptive Gradient Methods	6
2.4 Positioning Relative to the Reference Paper	6
Chapter 3. System Design and Methodology	7
3.1 Training Loop	7
3.2 Median Anchored Clipping	7
3.3 Server-Side Baselines	8
3.4 AdaGrad-OTA	8
3.5 Adam-OTA	9
3.6 One-Dimensional Intuition	9
3.7 Implementation Modules	10
Chapter 4. Experiments and Results	11
4.1 Data	11
4.2 Experiment Configuration	11
4.3 Evaluation Metrics	12
4.4 Main Method Comparison	13
4.5 Alpha-Stable Tail-Index Ablation	15
4.6 MAC Robust Pre-Processing	15
4.7 Client-Count Ablation	15
4.8 Discussion	16
Chapter 5. Conclusion and Future Work	18
5.1 Conclusion	18
5.2 Limitations	18
5.3 Future Work	19

References	20
Appendix	21
Author's Biography	22

Chapter 1. Introduction

1.1 Motivation

Federated learning (FL) trains a shared model across distributed clients without requiring the clients to upload their raw data. This design is attractive for edge intelligence because it reduces privacy exposure and keeps data near the devices that generate it. A standard FL round broadcasts the current global model, lets clients run local optimization, and aggregates their local updates on a central server. The FedAvg algorithm is the classic example of this workflow and remains the baseline for many FL systems [1].

The communication cost of conventional digital FL grows with the number of clients because each participating client must send an update to the server. Analog over-the-air aggregation changes the communication model. If clients transmit analog update waveforms at the same time, the wireless multiple-access channel physically superposes the signals. The server can then observe an aggregate update in one shared channel use instead of decoding each client separately. This makes OTA-FL a promising architecture for large-scale wireless learning.

The difficulty is that the same wireless channel also injects noise, interference, and possible fading. The server therefore receives a corrupted aggregate rather than the exact average update. When a plain FedAvg-style server update uses this noisy vector directly, heavy-tailed impulsive interference can cause slow convergence, oscillation, or divergence. The reference work “Adaptive Federated Learning Over the Air” studies this problem for adaptive methods under fading and alpha-stable interference [2]. This project follows that direction by making the alpha-stable tail index a core experimental variable; AWGN is treated as the special case $\alpha = 2$.

1.2 Problem Statement

This thesis studies FL over a simulated analog OTA aggregation channel. At communication round t , the server receives a noisy aggregate

$$\tilde{g}_t = \frac{1}{N} \left(\sum_{n=1}^N \Delta_{n,t} + \xi_t \right), \quad (1.1)$$

where N is the number of clients, $\Delta_{n,t}$ is the update produced by client n , and ξ_t is symmetric alpha-stable channel noise. Smaller α means heavier tails and more frequent impulsive outliers; $\alpha = 2$ recovers the Gaussian AWGN case. The central question is whether adaptive server optimizers can use their historical first- and second-moment states to make the training process more robust to this noisy aggregate.

The project should not be presented as a complete reproduction of Wang et al. [2]. The original paper contains a full convergence analysis and a broader experimental suite. The current codebase implements an empirical simulator for the core heavy-tail mechanism: alpha-stable OTA aggregation, adaptive server updates, and robust MAC pre-processing. Its contribution is a controlled, reproducible software validation rather than a new theoretical optimizer.

1.3 Contributions

The contributions of this project are as follows.

- A modular PyTorch simulator for OTA-FL with noisy analog aggregation, local client training, server-side optimization, logging, result export, and plotting.
- Implementations of FedAvg-OTA, FedAvgM-OTA, AdaGrad-OTA, and Adam-OTA as server-side optimizers that consume the OTA-aggregated update.
- Alpha-stable tail-index ablations that test how OTA-FL behaves as the channel changes from AWGN at $\alpha = 2$ to heavier-tailed interference at $\alpha < 2$.
- A MAC versus no-MAC comparison under heavy-tailed interference, testing whether

robust pre-processing complements adaptive server normalization.

- A lightweight one-dimensional theory sketch explaining why second-moment accumulation can induce automatic step-size shrinkage under impulsive channel noise.

1.4 Thesis Organization

Chapter 2 reviews FL, OTA aggregation, adaptive optimization, and the relationship between this project and the reference paper. Chapter 3 describes the implemented system model and optimizer equations. Chapter 4 presents the datasets, experimental settings, generated plots, and numerical results. Chapter 5 summarizes the outcome, limitations, and future work.

Chapter 2. Background and Related Work

2.1 Federated Learning

Federated learning considers the optimization of a global model over data distributed across multiple clients. If client n owns local data $\mathcal{D}_n$, the global objective can be written as

$$\min_w F(w) = \sum_{n=1}^N p_n F_n(w), \quad F_n(w) = \frac{1}{|\mathcal{D}_n|} \sum_{(x,y) \in \mathcal{D}_n} \ell(w; x, y), \quad (2.1)$$

where p_n is often proportional to the local data size. FedAvg trains this objective by alternating between local client optimization and server-side averaging [1]. The method is simple and communication efficient compared with fully synchronous distributed SGD, but it still requires explicit client uploads in each round.

2.2 Analog Over-the-Air Aggregation

OTA-FL replaces separate digital uploads with analog aggregation in the wireless channel. In an idealized channel, simultaneous client transmissions sum at the receiver, so the server observes an aggregate update directly. This property makes the required multiple-access resource less sensitive to the number of clients. However, the observed aggregate is noisy. A simplified channel model is

$$y_t = \sum_{n=1}^N \Delta_{n,t} + \xi_t, \quad (2.2)$$

and the server uses $\tilde{g}_t = y_t/N$ as the update direction. In the AWGN special case, $\xi_t \sim \mathcal{N}(0, \sigma^2 I)$. In the heavier channel model used by this project, ξ_t follows a symmetric alpha-stable distribution.

The original adaptive OTA-FL paper includes more general channel effects, including fading coefficients and symmetric alpha-stable interference [2]. This project does not implement fading or explicit power-control mechanisms, but it does include the alpha-stable interference axis. This is the key step beyond an AWGN-only simulator because alpha-stable noise captures rare but large impulsive channel perturbations.

2.3 Adaptive Gradient Methods

Adaptive optimizers normalize the update direction using historical gradient statistics. AdaGrad accumulates squared gradients and gives each coordinate a learning rate inversely proportional to the square root of its accumulated magnitude [3]. Adam combines first-moment smoothing, second-moment exponential averaging, and bias correction [4]. In centralized learning, these mechanisms are usually motivated by sparse gradients, noisy mini-batches, and heterogeneous coordinate scales.

In OTA-FL, the same mechanisms have an additional interpretation. If channel noise makes the aggregate update unusually large in some coordinate, the second-moment state for that coordinate increases. The denominator of the adaptive update then becomes larger, reducing the effective step size. This creates an implicit feedback loop from observed noisy update magnitude to server learning rate. The project tests whether this feedback loop is visible in training curves and ablation studies.

2.4 Positioning Relative to the Reference Paper

The reference paper proposes adaptive federated learning over the air and studies AdaGrad-like and Adam-like server updates with theoretical analysis under alpha-stable interference [2]. It evaluates multiple image and character classification tasks and includes studies over tail index, client count, and data heterogeneity.

The present project uses that paper as the conceptual source but narrows the deliverable. The implemented framework supports the core OTA-FL training loop, alpha-stable noise based on the Chambers-Mallows-Stuck method [5], MAC robust pre-processing, and adaptive server optimizers. The main evidence is the alpha tail-index ablation, with AWGN reported as the $\alpha = 2$ endpoint rather than the whole scope.

Chapter 3. System Design and Methodology

3.1 Training Loop

The simulator follows the standard synchronous FL round structure. At round t , the server broadcasts the global model w_t to all clients. Each client copies the global model, trains locally for E epochs using stochastic gradient descent, and returns a pseudo-gradient

$$\Delta_{n,t} = w_t - w_{n,t}^{(E)}. \quad (3.1)$$

This pseudo-gradient is the parameter difference between the broadcast model and the locally trained client model. It approximates the accumulated local descent direction.

The OTA oracle then aggregates the client pseudo-gradients. In the implemented alpha-stable setting,

$$\tilde{g}_t = \frac{1}{N} \left(\sum_{n=1}^N \Delta_{n,t} + \xi_t \right), \quad \xi_t \sim S\alpha S(\gamma), \quad (3.2)$$

where $S\alpha S(\gamma)$ denotes a symmetric alpha-stable random vector with tail index α and scale γ . The special case $\alpha = 2$ corresponds to AWGN with standard deviation $\sqrt{2}\gamma$ under the sampler convention. The optimizer receives $\tilde{g}_t$ and updates the global model in place.

3.2 Median Anchored Clipping

The simulator includes Median Anchored Clipping (MAC) as a robust pre-processing step before the server optimizer. For an aggregated vector $\tilde{g}_t$, MAC computes the coordinate median m , the median absolute deviation $r = \text{median}(|\tilde{g}_t - m|)$, and clips each coordinate to the interval

$$[m - kr, m + kr], \quad (3.3)$$

where k is the clipping factor. The default setting is $k = 3$. MAC is expected to have little effect in the Gaussian endpoint, but it can reduce rare impulsive coordinates when

$\alpha < 2$.

3.3 Server-Side Baselines

The plain OTA baseline applies the noisy aggregate directly:

$$w_{t+1} = w_t - \eta \tilde{g}_t. \quad (3.4)$$

The momentum baseline maintains a first-moment buffer:

$$m_t = \rho m_{t-1} + \tilde{g}_t, \quad w_{t+1} = w_t - \eta m_t. \quad (3.5)$$

These baselines are useful because they isolate whether the second-moment normalization in adaptive methods provides additional robustness beyond simple smoothing.

3.4 AdaGrad-OTA

AdaGrad-OTA maintains a smoothed update direction s_t and an accumulated coordinate-wise magnitude v_t :

$$s_t = \beta_1 s_{t-1} + (1 - \beta_1) \tilde{g}_t, \quad (3.6)$$

$$v_t = v_{t-1} + |s_t|^\alpha, \quad (3.7)$$

$$w_{t+1} = w_t - \eta \frac{s_t}{v_t^{1/\alpha} + \epsilon}. \quad (3.8)$$

For $\alpha = 2$, the denominator reduces to the usual square-root AdaGrad normalization. For $\alpha < 2$, the same form follows the fractional alpha-stable accumulation rule used in the reference paper. Because v_t is monotone increasing, AdaGrad-OTA tends to become conservative over long runs. This can stabilize noisy training, but it may also slow late-stage convergence.

3.5 Adam-OTA

The implemented Adam-OTA variant uses a stable server-side Adam update on the OTA-aggregated direction:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \tilde{g}_t, \quad (3.9)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) |\tilde{g}_t|^\alpha, \quad (3.10)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad (3.11)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad (3.12)$$

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\hat{v}_t^{1/\alpha} + \epsilon}. \quad (3.13)$$

When $\alpha = 2$, this is classical Adam applied at the server to the noisy OTA aggregate. For $\alpha < 2$, the second-moment update becomes a fractional moment accumulator. This preserves the project hypothesis because the update uses a state variable to compress effective step sizes when the observed aggregate has large magnitude.

3.6 One-Dimensional Intuition

The project does not reproduce the full proof in Wang et al. [2]. Instead, the core mechanism can be seen in a one-dimensional quadratic toy problem

$$f(w) = \frac{1}{2}(w - w^*)^2. \quad (3.14)$$

The noiseless gradient is $w_t - w^*$, while the OTA server observes

$$\tilde{g}_t = w_t - w^* + \xi_t, \quad \xi_t \sim S\alpha S(\gamma). \quad (3.15)$$

For an AdaGrad-style update,

$$v_t = v_{t-1} + |\tilde{g}_t|^\alpha, \quad w_{t+1} = w_t - \eta \frac{\tilde{g}_t}{v_t^{1/\alpha} + \epsilon}. \quad (3.16)$$

Whenever an impulsive noise sample makes $|\tilde{g}_t|$ large, it also makes v_t larger. The next effective step size

$$\eta_t^{\text{eff}} = \frac{\eta}{v_t^{1/\alpha} + \epsilon} \quad (3.17)$$

therefore contracts automatically. A fixed-step FedAvg-style update has no such feedback and can be dominated by rare heavy-tailed samples. The same mechanism also explains a limitation: if v_t grows too quickly, useful signal is compressed together with noise, which can slow training on harder tasks such as CIFAR-10. The rigorous multi-dimensional non-convex analysis is given by Wang et al. [2]; this section only provides an interpretable special case for the implemented simulator.

3.7 Implementation Modules

The codebase is organized around separable modules. The channel layer contains the noisy aggregation oracle and noise sampler. The data layer contains dataset loading and client partitioning. The model layer contains classification networks. The optimizer layer contains the four server optimizers. The experiment layer contains single-run training, method comparison, ablation, plotting, logging, and result export.

This organization supports the project goal in two ways. First, the alpha-stable experiments and MAC comparisons can be run reproducibly through command-line scripts. Second, the same interfaces can later be used to add richer physical channel models, additional datasets, or stricter reproduction settings without rewriting the training loop.

Chapter 4. Experiments and Results

4.1 Data

The experiments use the two datasets supported by the current codebase: MNIST and CIFAR-10. Both datasets are stored under the project-local `data/` directory and are loaded through the PyTorch `torchvision` dataset interfaces with `download=True`. No physical wireless-channel dataset is required, because the channel perturbation is generated synthetically by the OTA aggregation oracle.

MNIST is used as a lightweight sanity benchmark. It contains 60,000 training images and 10,000 test images of handwritten digits. The raw files are stored under `data/MNIST/raw/`. Images are converted to tensors and normalized with mean 0.1307 and standard deviation 0.3081.

CIFAR-10 is used as the main image-classification benchmark. It contains 50,000 training images and 10,000 test images across 10 object classes. The raw batch files `data_batch_1` through `data_batch_5`, `test_batch`, and `batches.meta` are stored under `data/cifar-10-batches-py/`. During training, CIFAR-10 uses random cropping with padding 4 and random horizontal flipping, followed by tensor conversion and channel-wise normalization. During testing, only tensor conversion and normalization are applied.

Federated clients are generated by partitioning the training set. The main comparison experiments use a non-IID Dirichlet partition with concentration 0.1 and random seed 42. In the generated partition, the MNIST comparison uses 10 clients, with client sizes ranging from 937 to 11,604 samples. The CIFAR-10 comparison uses 100 clients, with client sizes ranging from 12 to 1,733 samples. The single MNIST sanity run uses an IID split over 10 clients, with 6,000 samples per client.

4.2 Experiment Configuration

Table 4.1 summarizes the controlled comparison configuration. In these experiments all four server optimizers share the same dataset, model, partition, random seed, OTA

noise setting, and training budget. The OTA channel uses $\alpha = 2$, which corresponds to AWGN in the implemented sampler, and $\gamma = 0.05$.

Table 4.1 Controlled comparison configuration

Item	MNIST comparison	CIFAR-10 comparison
Dataset	MNIST	CIFAR-10
Model	MLP	ResNet-18
Optimizers	Four server methods	Four server methods
Number of clients N	10	100
Communication rounds	100	200
Local epochs E	5	5
Batch size	64	64
Server learning rate η	0.01	0.1
Local learning rate	0.01	0.01
Momentum / β_1	0.9	0.9
Adam β_2	0.999	0.999
Noise tail index α	2.0	2.0
Noise scale γ	0.05	0.05
Partition	non-IID Dirichlet	non-IID Dirichlet
Dirichlet concentration	0.1	0.1
Random seed	42	42

Table 4.2 summarizes the revised CIFAR-10 heavy-tail ablation configuration. The tail-index ablation varies α while fixing $\gamma = 0.05$. The MAC comparison uses the same heavy-tailed channel with $\alpha = 1.3$ and compares training with and without robust pre-processing.

4.3 Evaluation Metrics

The main metrics are test loss and test accuracy after each communication round. For comparison experiments, the final-round accuracy and the best accuracy achieved during the run are both reported. The final value measures the state of the trained model at the end of the fixed communication budget, while the best value helps reveal short-lived convergence peaks.

For the alpha ablation, the reported metric is final test accuracy under each tail index α . For the MAC comparison, the reported metric is final test accuracy with and without clipping. Multi-seed results are reported as mean $\pm$ standard deviation. All revised

Table 4.2 CIFAR-10 ablation configuration

Item	Value
Dataset / model	CIFAR-10 / ResNet-18
Optimizers in alpha study	FedAvg-OTA, FedAvgM-OTA, AdaGrad-OTA, Adam-OTA
Alpha values	1.1, 1.3, 1.5, 1.7, 1.9, 2.0
MAC comparison	no MAC vs MAC, $k = 3$
Communication rounds	100
Local epochs E	1
Batch size	64
Server learning rate η	0.01
Local learning rate	0.01
Momentum / β_1	0.9
Adam β_2	0.999
Noise scale γ	0.05
Partition	non-IID Dirichlet, concentration 0.1
Random seeds	42, 43, 44

heavy-tail values are generated from the JSON files under `results/heavytail/` rather than copied from a single run.

4.4 Main Method Comparison

The MNIST comparison in Table 4.3 is used as a sanity benchmark for adaptive server-side normalization. FedAvg-OTA reaches 67.85% final accuracy, while FedAvgM-OTA improves to 89.24%. AdaGrad-OTA and Adam-OTA reach 93.05% and 93.46%, respectively. This supports the hypothesis that second-moment adaptation can stabilize OTA training in this lighter benchmark, but it should not be treated as the main heavy-tail evidence.

Table 4.3 MNIST comparison results, $N = 10$, $\gamma = 0.05$

Method	Final loss	Final accuracy	Best accuracy
FedAvg-OTA	1.5018	67.85%	67.85%
FedAvgM-OTA	0.3498	89.24%	89.24%
AdaGrad-OTA	0.2091	93.05%	93.19%
Adam-OTA	0.2227	93.46%	93.46%

The CIFAR-10 comparison in Table 4.4 is more nuanced. FedAvgM-OTA is the strongest method in the controlled 200-round run, reaching 64.42% final accuracy. AdaGrad-

OTA and Adam-OTA improve over plain FedAvg-OTA, but they do not exceed the momentum baseline under this specific CIFAR-10 configuration. Therefore, the CIFAR-10 result should not be stated as “Adam-OTA is always best”. A more accurate conclusion is that adaptive methods help relative to FedAvg-OTA, while FedAvgM-OTA remains highly competitive when its momentum is well matched to the task. This AWGN endpoint motivates the heavier alpha-stable experiments below: under Gaussian noise alone, momentum can already be sufficient, so the project’s distinctive question is whether adaptive normalization and MAC help when the channel has impulsive tails.

Table 4.4 CIFAR-10 comparison results, $N = 100$, $\gamma = 0.05$

Method	Final loss	Final accuracy	Best accuracy
FedAvg-OTA	1.6167	39.23%	39.23%
FedAvgM-OTA	1.0087	64.42%	64.42%
AdaGrad-OTA	1.5217	43.11%	43.58%
Adam-OTA	1.6262	41.29%	41.64%

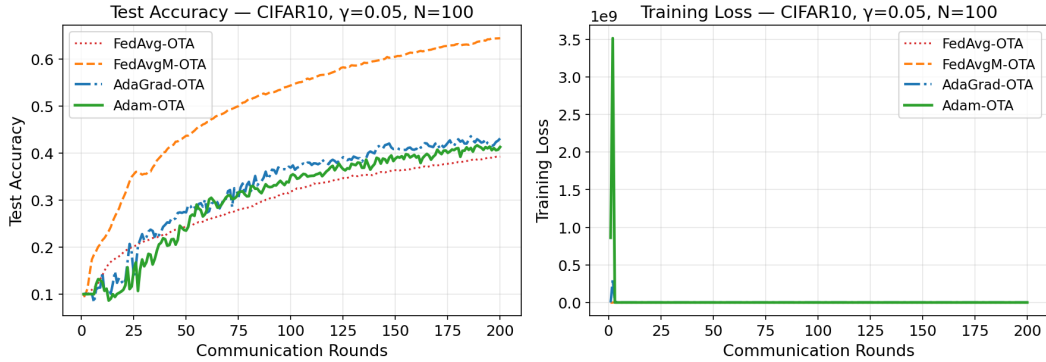


Figure 4.1 CIFAR-10 comparison curves for FedAvg-OTA, FedAvgM-OTA, AdaGrad-OTA, and Adam-OTA.

The single-run outputs provide an additional sanity check, but they are not a controlled four-method comparison because their hyperparameters differ from Table 4.1. The single CIFAR-10 Adam-OTA run uses batch size 128 and server learning rate 0.01, reaching 61.51% final accuracy. The single MNIST AdaGrad-OTA run uses an IID client split and reaches 97.57% final accuracy. These runs confirm that the implemented optimizers can train successfully, but the main conclusions should be drawn from the controlled comparison tables above.

4.5 Alpha-Stable Tail-Index Ablation

The revised core experiment varies the alpha-stable tail index instead of only varying the AWGN scale. The endpoint $\alpha = 2$ is Gaussian AWGN, while smaller values of α produce heavier-tailed impulsive interference. This is the setting where adaptive normalization is most meaningful: a server method must handle rare but very large aggregate perturbations.

Table 4.5 CIFAR-10 alpha-stable tail-index ablation

Output	Status
Alpha ablation	Run the alpha ablation module, then regenerate tables and figures.

Alpha-ablation figure is generated by `./run_all.sh` after the heavy-tail runs finish.

Figure 4.2 CIFAR-10 final accuracy as a function of alpha-stable tail index α .

The expected qualitative pattern is that convergence becomes slower and less stable as α decreases. The adaptive methods should be interpreted through this heavy-tail curve rather than through an AWGN-only sweep over γ .

4.6 MAC Robust Pre-Processing

The second revised experiment compares training with and without Median Anchored Clipping under a heavy-tailed setting. The default comparison uses $\alpha = 1.3$, $\gamma = 0.05$, and clipping factor $k = 3$. MAC is not expected to change the Gaussian endpoint dramatically, but it should reduce the effect of rare impulsive coordinates when $\alpha < 2$.

Table 4.6 CIFAR-10 MAC comparison under heavy-tailed interference

Output	Status
MAC comparison	Run the MAC comparison module, then regenerate tables and figures.

MAC comparison figure is generated by `./run_all.sh` after the heavy-tail runs finish.

Figure 4.3 CIFAR-10 comparison of no-MAC and MAC under alpha-stable interference.

4.7 Client-Count Ablation

Table 4.7 reports the Adam-OTA client-count ablation on CIFAR-10. With the same 100-round budget, increasing the number of clients improves final accuracy from 10.26% at 5 clients to 21.72% at 100 clients. This trend is consistent with OTA aggregation: averaging more client updates can make the aggregate direction more informative, even though the data remain non-IID and the channel still injects noise.

Table 4.7 CIFAR-10 client-count ablation with Adam-OTA

Clients N	Final loss	Final accuracy	Best accuracy
5	2.3121	10.26%	10.36%
10	2.2288	16.92%	16.96%
20	2.1249	19.99%	19.99%
50	2.1171	20.36%	20.36%
100	2.0880	21.72%	21.72%

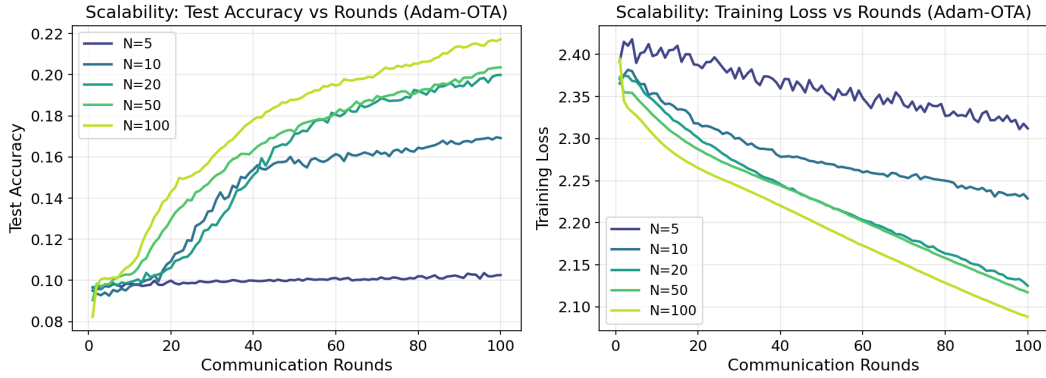


Figure 4.4 CIFAR-10 client-count learning curves for Adam-OTA.

4.8 Discussion

The experiments should be interpreted in two layers. The existing AWGN comparison remains a useful endpoint sanity check: it shows that the implementation trains successfully and that FedAvgM-OTA is a strong CIFAR-10 baseline. The revised alpha-stable

ablation is the central evidence for the project claim, because it tests the heavy-tailed channel condition that motivates adaptive OTA-FL in the first place.

The project conclusion should therefore be phrased carefully. The results do not prove that Adam-OTA universally dominates all baselines. Instead, the claim is that fractional second-moment adaptive updates and robust pre-processing provide practical mechanisms for improving OTA-FL stability when the aggregate update is corrupted by alpha-stable impulsive noise. If adaptive methods underperform FedAvgM-OTA in some CIFAR-10 settings, that is not a contradiction; it indicates that aggressive normalization can compress useful signal as well as channel noise.

Chapter 5. Conclusion and Future Work

5.1 Conclusion

This thesis presents an empirical framework for adaptive federated learning over the air under alpha-stable channel interference. The project is motivated by the tension between OTA-FL’s communication scalability and its noisy aggregation channel. The implemented system tests whether server-side adaptive optimization can act as an implicit, CSI-free noise-mitigation mechanism through its fractional second-moment accumulator.

The completed AWGN endpoint experiments show that this mechanism is useful but not universally dominant. On MNIST, AdaGrad-OTA and Adam-OTA achieve the best controlled-comparison accuracies, reaching 93.05% and 93.46% respectively. On CIFAR-10, FedAvgM-OTA is the strongest method in the 200-round AWGN comparison, with 64.42% final accuracy, while AdaGrad-OTA and Adam-OTA still improve over plain FedAvg-OTA. This motivates the revised heavy-tail experiments: the core claim should be evaluated under alpha-stable impulsive interference, not only under the Gaussian endpoint.

The main output is therefore not a complete theoretical proof or a full reproduction of all experiments in the reference paper. Instead, the output is a working PyTorch simulator, alpha-stable tail-index experiments, MAC robustness comparisons, and a set of figures and JSON logs that support a carefully scoped adaptive OTA-FL robustness claim.

5.2 Limitations

The project remains narrower than the full reference paper in three main ways. First, it provides only a toy theoretical explanation and points to the reference paper for the complete non-convex convergence proof. Second, the wireless channel is still a simulation oracle and does not model mobility, multi-cell interference, synchronization, CSI acquisition, or power control. Third, the project does not include a software-defined-radio or

real wireless testbed validation.

5.3 Future Work

Future extensions can make the project closer to a full wireless learning system by adding richer channel dynamics, studying mobility and power-control effects, and evaluating the method on a real software-defined-radio testbed. A theoretical extension could replace the toy analysis with a complete project-specific proof for the implemented optimizer variants.

References

- [1] McMahan B, Moore E, Ramage D, et al. Communication-efficient learning of deep networks from decentralized data[C]//Proceedings of the 20th International Conference on Artificial Intelligence and Statistics. PMLR, 2017: 1273-1282.
- [2] Wang C, Chen Z, Pappas N, et al. Adaptive federated learning over the air[A]. 2024.
- [3] Duchi J, Hazan E, Singer Y. Adaptive subgradient methods for online learning and stochastic optimization[J]. Journal of Machine Learning Research, 2011, 12(61): 2121-2159.
- [4] Kingma D P, Ba J. Adam: A method for stochastic optimization[C]//International Conference on Learning Representations. 2015.
- [5] Chambers J M, Mallows C L, Stuck B W. A method for simulating stable random variables[J]. Journal of the American Statistical Association, 1976, 71(354): 340-344.

Appendix

(Supplementary Materials)

The supplementary materials for this thesis are maintained in the project repository. The main experiment commands are collected in `run_all.sh` and `RUN.md`. The revised heavy-tail runs are stored under `results/heavytail/`. Generated figures are stored under `results/figures/`, and generated LaTeX tables are stored under `template/data/generated/`. These files provide the direct record of the runs reported in the experimental section.

Author's Biography

Peiran Wei is an undergraduate student in Electrical and Computer Engineering at the Zhejiang University-University of Illinois Urbana-Champaign Institute. The author's academic interests include federated learning, wireless communication systems, and machine learning systems for edge intelligence.